CẨM NANG 10 KỸ THUẬT THIẾT KẾ THUẬT TOÁN VÀ TỐI ƯU HÓA HỆ THỐNG ĐỈNH CAO ĐÚC KẾT TỪ reference_solve.cpp

Các Mẫu Thiết kế (Design Patterns) Kinh điển cho Bài toán Định tuyến (VRP), Lập lịch (Scheduling) và Tối ưu Tổ hợp Thực chiến

Tài liệu Tổng hợp Kỹ thuật Chuyên sâu — Chuyển hóa Tư duy từ Metaheuristics sang Matheuristics

---

Mục lục

1 Kỹ thuật 1: Tư duy "Đảo chiều NP-Hard" — Khi Ràng buộc Chặt Biến Vét cạn Thành Vũ khí Tối thượng **2**
- 1.1 Bẫy Tư duy Thông thường và Quy luật Đối ngẫu của Ràng buộc 2
- 1.2 Mẫu Thiết kế "Exact-First with Safe Fallback" (Thử Vét cạn Trước, Dự phòng Metaheuristic Sau) 2

2 Kỹ thuật 2: Đồng nhất hóa Thực thể bằng Phép Co Đồ thị (Macro-Node Contraction) **3**
- 2.1 Vấn đề của Đồ thị Không đồng nhất (Heterogeneous Tasks) 3
- 2.2 Giải pháp: Đóng gói Chuỗi Hành động Bắt buộc thành Nguyên tử (Macro-Node) . . 3

3 Kỹ thuật 3: Cắt tỉa Thống trị Đẳng cấu Ngay trong Lúc Duyệt bằng Băm 64-bit (enum_record) **3**
- 3.1 Nguyên lý Tách "Giao diện Ngoài" khỏi "Cấu trúc Nội bộ" 3

4 Kỹ thuật 4: Thư giãn Đối ngẫu Lagrangian + Bước nhảy Polyak như một "La bàn Cận trên" Siêu tốc **4**
- 4.1 Tại sao Mọi Thuật toán Heuristic Đỉnh cao Đều Cần Một Cận trên Nội tại (UB)? . 4

5 Kỹ thuật 5: Toán tử Chuỗi Đẩy-và-Lấp Tham lam trên Danh sách Sắp sẵn (Eject-and-Refill) **4**

6 Kỹ thuật 6: Quy hoạch Động Hai chiều Tiền tố dep[] + Hậu tố lat[] và Ngắt sớm Đồng bộ (sync) **5**

7 Kỹ thuật 7: Bộ nhớ đệm Gia tăng theo Số Phiên bản (Versioned Incremental Cache) **5**

8 Kỹ thuật 8: Sao lưu Lười theo Nhãn Thời gian & Hoàn tác Nhật ký (Timestamped Lazy Backup & Undo Log) **5**

9 Kỹ thuật 9: Chế độ Nhiệt độ Tự Chuẩn hóa theo Thang đo, Tiết lưu Đồng hồ Bitmask & Kỹ thuật Nháy mắt (Blink) **6**

10 Kỹ thuật 10: Tối ưu hóa Cấp độ Vi kiến trúc CPU (Flat Arena, Pointer Hoisting, Xorshift64 & Fast I/O) **6**

1 Kỹ thuật 1: Tư duy "Đảo chiều NP-Hard" — Khi Ràng buộc Chặt Biến Vét cạn Thành Vũ khí Tối thượng

1.1 Bẫy Tư duy Thông thường và Quy luật Đối ngẫu của Ràng buộc

Khi đối mặt với một bài toán NP-Hard có kích thước $R = 500$ yêu cầu và $K = 125$ xe, phản xạ tự nhiên của hầu hết lập trình viên là bỏ qua ngay phương án duyệt nhánh-cận (DFS/Branch-and-Bound) vì sợ bùng nổ tổ hợp $O(R!)$, và lao vào viết Local Search / Tabu Search / ALNS.

Tuy nhiên, `reference_solve.cpp` dạy chúng ta một bài học kinh điển trong Operations Research:

Nguyên lý "Lời nguyền cũng là Phúc lành" (The Curse & Blessing of Tight Constraints)

Độ phức tạp của việc liệt kê toàn bộ lộ trình hợp lệ của một chiếc xe **không phụ thuộc vào tổng số yêu cầu $R = 500$ hay tổng số xe $K = 125$**, mà phụ thuộc vào **số lượng yêu cầu tối đa mà 1 chiếc xe có thể chở trong 1 ca làm việc ($L_{\max}$)**:

- Khi ràng buộc lỏng ($L_{\max} \geq 15$): Không gian nghiệm cực kỳ trơn tru $\implies$ Local Search / Tabu Search hoạt động rất tốt, còn DFS sinh cột bị nổ tổ hợp.
- Khi ràng buộc chặt ($L_{\max} \leq 5$, do Time Window hẹp hoặc tải trọng nhỏ): Không gian nghiệm lởm chởm như bãi mìn làm Local Search bị kẹt và dao động mạnh ($\sim 3\%$), **NHƯNG số lộ trình hợp lệ của mỗi xe lại co nhỏ về chỉ còn vài trăm đến vài nghìn!** Lúc này, vét cạn toàn bộ lộ trình từng xe rồi giải **Set Packing** sẽ đập tan bài toán trong vài mili-giây!

1.2 Mẫu Thiết kế "Exact-First with Safe Fallback" (Thử Vét cạn Trước, Dự phòng Metaheuristic Sau)

Đừng bao giờ đoán mò xem bộ test của giám khảo có ràng buộc chặt hay lỏng — hãy để chương trình **tự động thử vét cạn trước với một cầu chì an toàn (Safety Fuse)**:

```
static const long long ENUM_STATE_CAP = 5000000;
static const size_t ENUM_COL_CAP = 2000000;
static const double ENUM_TIME_FRAC = 0.25; // Chi danh toi da 25% thoi gian de thu vet can

static void enum_dfs(...) {
    if (enum_abort) return;
    if (++enum_states > ENUM_STATE_CAP || cols.size() >= ENUM_COL_CAP) { enum_abort = true; return; }
    if ((enum_states & 4095) == 0 && steady_clock::now() > enum_deadline) {
        enum_abort = true; return; }
    ...
}
```

Bài học áp dụng: Trong mọi bài toán phân bổ / lập lịch / định tuyến nhiều máy (multi-vehicle / multi-machine), hãy luôn viết một hàm `column_solve()` đặt ở đầu hàm `main`. Nếu `enum_abort == false`, bạn ăn trọn điểm tối ưu tuyệt đối; nếu `enum_abort == true`, bạn giải phóng bộ nhớ và rơi xuống Phase 2 (Tabu/SISR) mà chỉ mất tối đa 25% thời gian!

—

2 Kỹ thuật 2: Đồng nhất hóa Thực thể bằng Phép Co Đồ thị (Macro-Node Contraction)

2.1 Vấn đề của Đồ thị Không đồng nhất (Heterogeneous Tasks)

Trong bài toán SARP, khách hàng (Passenger) và hàng hóa (Parcel) có quy tắc đi lại khác nhau: khách lên xe tại P_i là phải đi thẳng tới D_i ngay lập tức, trong khi hàng hóa lên tại P_j có thể nằm trên xe qua nhiều điểm dừng rồi mới trả tại D_j. Nếu biểu diễn cả hai bằng các đỉnh rời rạc giống nhau, mọi hàm kiểm tra chèn và duyệt DFS đều phải phân nhánh `if (is_passenger) ... else ...` cực kỳ rối rắm và dễ sai sót.

2.2 Giải pháp: Đóng gói Chuỗi Hành động Bắt buộc thành Nguyên tử (`Macro-Node`)

Bất kỳ chuỗi sự kiện nào bắt buộc phải thực hiện liền kề nhau không ngắt quãng ($P_r \to D_r$) đều có thể **co lại (contract) thành 1 đỉnh tác vụ duy nhất** x được đặc trưng bởi 5 thuộc tính chuẩn hóa:

$$\big(\text{nin}[x],\ \text{nout}[x],\ \text{ndur}[x],\ \text{nic}[x],\ \text{nw}[x]\big).$$

- Với khách hàng $r \le N$: $\text{nin}[r] = P_r$, $\text{nout}[r] = D_r$, $\text{ndur}[r] = 2S_r + T(P_r, D_r)$, $\text{nic}[r] = C(P_r, D_r)$, $\text{nw}[r] = 0$.
- Với điểm đón/trả hàng: $\text{nin}[x] = \text{nout}[x] =$ đỉnh vật lý, $\text{ndur}[x] = S_r$, $\text{nic}[x] = 0$, $\text{nw}[x] = \pm W_r$.

Lợi ích kép:

1. Giảm một nửa độ dài mảng lộ trình cho tất cả các tác vụ chở khách (chèn khách chỉ cần chọn 1 vị trí i thay vì cặp (i, j)).
2. Mọi phép tính thời gian đến $\text{arr} = \text{dep}_{\text{prev}} + T(\text{nout}_{\text{prev}}, \text{nin}[x])$ và thời gian rời $\text{dep} = \max(\text{arr}, ne[x]) + \text{ndur}[x]$ trở nên **đồng nhất 100%** cho cả khách lẫn hàng!

—

3 Kỹ thuật 3: Cắt tỉa Thống trị Đẳng cấu Ngay trong Lúc Duyệt bằng Băm 64-bit (`enum_record`)

3.1 Nguyên lý Tách "Giao diện Ngoài" khỏi "Cấu trúc Nội bộ"

Khi sinh các phương án con cho một đơn vị (một xe, một máy, một ca trực), hãy tự hỏi: *"Phần còn lại của hệ thống cần biết thông tin gì về phương án con này?"*

- Phần còn lại của đội xe **không quan tâm** xe k đi theo thứ tự nào bên trong lộ trình (`e_seq`).
- Phần còn lại của đội xe **chỉ quan tâm**: Xe nào chạy (k), tập yêu cầu nào đã bị tiêu thụ ($R_j = \text{sorted}(\texttt{e_reqs})$), và đem lại bao nhiêu tiền (profit).

Do đó, mọi lộ trình có cùng bộ khóa ngoại (k, R_j) đều thuộc cùng một **lớp tương đương đẳng cấu (Isomorphism Class)**. Ta dùng hàm băm 64-bit kết hợp FNV-1a và hằng số tỉ lệ vàng `0x9E3779B97F4A7C15ULL` để ánh xạ $(k, R_j) \mapsto h \in \mathbb{Z}_{2^{64}}$ và chỉ giữ lại đại diện có profit lớn nhất!

```
uint64_t h = 14695981039346656037ull ^ (uint64_t)k * 0x9E3779B97F4A7C15ull;
for (int q : e_sorted) {
    h ^= (uint64_t)q + 0x9E3779B97F4A7C15ull + (h << 6) + (h >> 2);
    h *= 1099511628211ull;
}
```

Bài học áp dụng: Kỹ thuật băm tập hợp đã sắp xếp (Canonical Set Hashing) này nhanh gấp ≈ 15 lần so với việc dùng `std::map<vector<int>, int>` và giúp cắt giảm $65\% \sim 80\%$ số lượng cột trùng lặp.

—

4 Kỹ thuật 4: Thư giãn Đối ngẫu Lagrangian + Bước nhảy Polyak như một "La bàn Cận trên" Siêu tốc

4.1 Tại sao Mọi Thuật toán Heuristic Đỉnh cao Đều Cần Một Cận trên Nội tại (UB)?

Khi chạy Tabu Search hay SA thông thường, bạn luôn mù mờ trước câu hỏi: *"Điểm số* $13,279$ *hiện tại đã tối ưu chưa, hay vẫn còn có thể lên* $14,000$*?"* Chỉ với **30 dòng code** (Lines 462–497), tác giả đã nhúng một bộ giải **Lagrangian Relaxation + Subgradient Optimization** ngay trước vòng lặp SA:

1. Gán nhân tử phạt $u_q \geq 0$ cho mỗi ràng buộc bị tranh chấp giữa các xe ($\sum_{j \ni q} x_j \leq 1$).
2. Tại mỗi bước, tính tổng lợi nhuận hiệu chỉnh tốt nhất của từng xe:
$$L(u) = \sum_q u_q + \sum_{k=1}^{K} \max_{j \in \mathcal{C}_k} \left(0,\ \text{profit}_j - \sum_{q \in R_j} u_q \right).$$
3. Đếm số lần mỗi yêu cầu q bị chọn trùng m_q, tính đạo hàm $g_q = 1 - m_q$, và cập nhật giá phạt theo công thức Polyak:
$$u_q \leftarrow \max\left(0,\ u_q - \lambda \frac{L(u) - \text{best}}{\|g\|_2^2} g_q\right).$$

Ba Lợi ích Vô giá của 30 Dòng Code Lagrangian

1. **Chứng nhận Dừng sớm Tức thì (Zero-Waste Early Stopping)**: Ngay khi SA tìm được nghiệm có best $== \lfloor L(u) \rfloor$, chương trình lập tức `break` và kết thúc ở mili-giây thứ 20 (như trên 6/8 bộ test)!
2. **Bắt Nghiệm Nguyên Tối ưu Không cần SA**: Nếu tại vòng lặp bất kỳ mà $\|g\|_2^2 == 0$ (Complementary Slackness), nghiệm thư giãn `relax[k]` tự động là nghiệm nguyên tối ưu toàn cục (như trên `test_65` và `test_106`).
3. **Không cần thư viện ngoài**: Không cần cài đặt OR-Tools, CPLEX hay Gurobi mà vẫn tính được cận trên $\lfloor OPT_{LP} \rfloor$ chính xác tới từng đơn vị chỉ trong 2 ms!

—

5 Kỹ thuật 5: Toán tử Chuỗi Đẩy-và-Lấp Tham lam trên Danh sách Sắp sẵn (`Eject-and-Refill`)

Khi giải bài toán phân bổ hoặc đóng gói (Set Packing / Resource Allocation), thay vì chỉ đổi phương án của 1 cá thể trong phần tài nguyên còn trống, hãy dùng mẫu thiết kế **Eject-and-Refill**:

1. **Tiền xử lý 1 lần**: Sắp xếp danh sách phương án `tcols[k]` của từng cá thể k theo lợi nhuận giảm dần.
2. **Cưỡng chế Đẩy (Force & Eject)**: Cứ gán phương án mới j cho cá thể k; bất kỳ cá thể k_2 nào đang giữ tài nguyên bị đụng độ với j sẽ bị tháo phương án ra và đẩy vào mảng `refill`.
3. **Lấp lại Đầu mảng (First-Fit Sorted Refill)**: Với mỗi $k_2 \in$ `refill`, quét từ đầu mảng `tcols[k2]` và chọn ngay phương án đầu tiên không xung đột (`is_free(c)`). Nhờ mảng đã sắp giảm dần từ trước, phần tử đầu tiên thỏa mãn chính là phương án tốt nhất tuyệt đối của k_2!

—

6 Kỹ thuật 6: Quy hoạch Động Hai chiều Tiền tố dep[] + Hậu tố lat[] và Ngắt sớm Đồng bộ (sync)

Để đánh giá việc chèn một phần tử vào giữa lộ trình có n điểm dừng trong thời gian $O(1)$ thay vì $O(n)$:

1. **Lưu Tiền tố Thời gian Rời Sớm nhất** (`dep[i]`): Tính xuôi từ $1 \to n$.
2. **Lưu Hậu tố Thời điểm Đến Muộn nhất** (`lat[i]`): Tính ngược từ $n \to 1$ theo công thức:

$$\text{lat}[i] = \min\Big(nl[\text{nd}[i]],\ \text{lat}[i+1] - T(\text{nout}[\text{nd}[i]], \text{nin}[\text{nd}[i+1]]) - \text{ndur}[\text{nd}[i]]\Big).$$

   Khi đó, mọi phép chèn làm thời điểm đến đỉnh $i+1$ mới là t_{i+1}^{new} chỉ cần kiểm tra đúng 1 điều kiện $t_{i+1}^{\text{new}} \le \text{lat}[i+1]$ trong $\mathbf{O(1)}$ để biết chắc chắn toàn bộ phần đuôi $i+1 \dots n$ hợp lệ!
3. **Ngắt sớm Đồng bộ Thời gian** (`if (dj == dep[j]) sync = true`): Khi cần lan truyền độ trễ qua một đoạn $[i+1, j]$ (như khi chèn cặp Pickup-Dropoff), ngay khi gặp một đỉnh có thời gian chờ hấp thụ hết độ trễ làm thời gian rời mới bằng thời gian rời cũ (dj == dep[j]), lập tức bật cờ `sync = true` và dừng lại toàn bộ giá trị dep[] cũ cho các đỉnh phía sau!

---

7 Kỹ thuật 7: Bộ nhớ đệm Gia tăng theo Số Phiên bản (Versioned Incremental Cache)

Trong các thuật toán LNS / Ruin & Recreate / Tabu Search, ở mỗi vòng lặp chỉ có một số rất nhỏ lộ trình bị thay đổi ($2 \sim 4$ xe trên tổng số 125 xe), nhưng bước Recreate lại cần tìm vị trí chèn tốt nhất của yêu cầu q trên toàn bộ 125 xe.

```
static vector<int> rver;  // Phien ban hien tai cua xe k
static int gver = 0;      // Bo dem phien ban toan cuc
struct CacheEntry { int ver = -1, i = 0, j = 0; ll cost = 0; };
static vector<CacheEntry> icache; // Mang phang [q * (K + 1) + k]

// Khi xe k thay doi (trong recompute hoac rollback):
rver[k] = ++gver;

// Khi can tim vi tri chen q vao xe k:
CacheEntry& e = icache[(size_t)q * (K + 1) + k];
if (e.ver != rver[k]) {
    e.cost = best_insert(k, q, e.i, e.j, blink);
    e.ver = rver[k];
}
```

Ưu điểm vượt trội: Không bao giờ phải duyệt xóa (invalidate) bảng cache khi một xe thay đổi! Chỉ cần tăng `rver`$[k]$ = `++gver` trong $O(1)$, mọi ô nhớ cache cũ của xe k tự động hết hạn, trong khi $> 96\%$ ô nhớ cache của các xe không bị chạm vẫn giữ nguyên hiệu lực!

---

8 Kỹ thuật 8: Sao lưu Lười theo Nhãn Thời gian & Hoàn tác Nhật ký (Timestamped Lazy Backup & Undo Log)

Trong một vòng lặp chạy 10^6 lần/giây, việc gọi `memset`, `std::fill` hoặc sao chép toàn bộ nghiệm `backup_sol = cur_sol` trước mỗi bước thử sẽ làm tốc độ tụt giảm hàng chục lần. `reference_solve.cpp` giải quyết bằng 2 cơ chế:

1. **Mảng Đánh dấu Nhãn Thời gian (`stamp_id`) thay cho `memset`**: Mỗi vòng lặp chỉ tăng `++stamp_id`. Muốn kiểm tra xem xe k đã được sao lưu trong vòng lặp hiện tại chưa, chỉ cần kiểm tra `touch_stamp`$[k]$ == `stamp_id`:

```
static inline void touch(int k) {
    if (touch_stamp[k] != stamp_id) {
        touch_stamp[k] = stamp_id;
        touched.push_back(k);
        backup[k] = rt[k]; // Chi sao luu dung chiec xe sap bi sua!
    }
}
```

2. **Hoàn tác bằng Phép Hoán đổi Con trỏ `swap()` và Nhật ký Ngược**: Nếu bước nhảy SA bị từ chối, chỉ cần duyệt qua đúng vài chiếc xe trong `touched` và gọi `swap(rt[k], backup[k])` (thao tác $O(1)$ hoán đổi con trỏ bộ nhớ trong của `std::vector` mà không cấp phát hay chép lại mảng!), kết hợp duyệt ngược `veh_log` để khôi phục mảng `veh[r]`:

```
for (int k : touched) { swap(rt[k], backup[k]); rver[k] = ++gver; }
for (int i = (int)veh_log.size() - 1; i >= 0; --i) veh[veh_log[i].first] =
    veh_log[i].second;
```

—

9 Kỹ thuật 9: Chế độ Nhiệt độ Tự Chuẩn hóa theo Thang đo, Tiết lưu Đồng hồ Bitmask & Kỹ thuật Nháy mắt (`Blink`)

1. **Tự chuẩn hóa Nhiệt độ theo Dữ liệu Đầu vào (Scale-Invariant Temperature)**: Luôn đặt $T_0 = \alpha \cdot \overline{\text{Profit}}$ và $T_F = \beta \cdot \overline{\text{Profit}}$ (ví dụ $\alpha = 0.3 \sim 1.0, \beta = 0.003 \sim 0.005$ nhân với doanh thu trung bình của 1 yêu cầu hoặc 1 cột). Nhờ đó bộ tham số SA hoạt động chuẩn xác trên mọi bộ dữ liệu dù đơn vị tiền tệ là 10 hay 10^9.

2. **Tiết lưu Lệnh gọi Đồng hồ Hệ thống bằng Bitmask (`Clock Throttling`)**: Lệnh `chrono::steady_clock` và hàm mũ `pow()` tốn khoảng 25 $\sim$ 40 ns. Trong vòng lặp siêu nhanh, chỉ gọi đồng hồ khi `(it & 255) == 0` (mỗi 256 vòng lặp một lần), giúp tiết kiệm hơn 99.6% chi phí hệ thống!

3. **Biến đổi Logarit Tiêu chuẩn Metropolis**: Viết `delta >= 0 || (double)delta > temp * log(rand01() + 1e-300)` để tận dụng đoản mạch khi $\Delta \geq 0$ và tránh tràn số dưới (underflow) của hàm `exp(delta / temp)`.

4. **Nhiễu Nháy mắt Christiaens (`BLINK_MASK = 63`)**: Trong hàm tìm vị trí chèn tốt nhất, thêm điều kiện `!(blink && (rng() & 63) == 0)` để bỏ qua ứng viên tốt nhất với xác suất $1/64 \approx 1.56\%$. Chỉ tốn 1 phép `AND` bit nhưng tạo ra xác suất $\approx 12\%$ trên một chuỗi 8 yêu cầu để tự động chuyển hướng sang vị trí tốt thứ hai (Stochastic Regret-2), chống lặp vô hạn hoàn hảo!

—

10 Kỹ thuật 10: Tối ưu hóa Cấp độ Vi kiến trúc CPU (Flat Arena, Pointer Hoisting, Xorshift64 & Fast I/O)

Bốn kỹ thuật lập trình hệ thống giúp `reference_solve.cpp` đạt tốc độ tiệm cận giới hạn phần cứng:

1. **Cấp phát Vùng nhớ Phẳng (Flat Arena Allocation)**: Thay vì khai báo `struct Column { vector<int> reqs, nodes; }` (mỗi cột tốn 2 lần cấp phát heap rời rạc gây trượt bộ nhớ đệm L1/L2 Cache Miss), hãy gom toàn bộ dữ liệu của mọi cột vào 2 mảng phẳng toàn cục `col_req` và `col_nodes`, còn mỗi `Column` chỉ lưu chỉ số bắt đầu và độ dài `(rbeg, rlen, nbeg, nlen)`.

2. **Kéo Con trỏ Thô ra Ngoài Vòng lặp Nóng (Raw Pointer Hoisting)**: Ở đầu hàm `best_insert` (Line 130), trước khi bước vào vòng lặp kép, tác giả lấy toàn bộ con trỏ thô:

```
const int* nd = r.nd.data(); const ll* dep = r.dep.data();
const ll* lat = r.lat.data(); const int* ld = r.ld.data();
```

   Giúp trình biên dịch GCC vector hóa và giữ địa chỉ mảng trực tiếp trong thanh ghi CPU.

3. **Bộ sinh Số ngẫu nhiên 64-bit Xorshift Siêu nhẹ (Lines 58–61)**: Thay vì dùng `std::mt19937_64` (nặng 2.5 KB trạng thái), chỉ dùng đúng 1 thanh ghi 64-bit và 4 chu kỳ lệnh:

```
static uint64_t rng_s = 0x9E3779B97F4A7C15ull;
static inline uint64_t rng() { rng_s ^= rng_s << 7; rng_s ^= rng_s >> 9; return
    rng_s; }
static inline double rand01() { return (rng() >> 11) * (1.0 /
    9007199254740992.0); }
```

4. **Đọc Toàn bộ Input bằng `fread` một Lần Duy nhất (Lines 28–53)**: Đọc trọn vẹn file đề bài (1.6 MB của `test_500.txt`) vào bộ đệm `ibuf` chỉ trong < 0.5 ms, tự động bỏ qua các dòng chú thích `'#'`, dành trọn 99.95% quỹ thời gian cho thuật toán tối ưu!